It's probably worth knowing that PHP used to have some serious vulnerabilities concerning null bytes in strings; most have been fixed since PHP 5.3, but be aware that if a null byte exists in the middle of string data, it will potentially be truncated beyond that byte. There are also techniques to bypass the protections that have been added to PHP. Penetration tests are recommended, as is input validation.

Strict Types

As mentioned in the previous chapter, it is possible to force the PHP engine to be strict about scalar types and prevent type coercion on calls. If a `strict_types` declaration is placed at the top of the source code file, the remaining code in that file will be subjected to stricter rules of scalar type checking when calls are made. One thing to note is that strict type checking is on a per-file basis; the declaration has to be added to all source code files where strictness is required:

```
declare(strict_types=1); // this MUST be the first statement in the
                         source file!

$num = 123;
$res = trim($num);

Output:
Fatal error: Uncaught TypeError: trim(): Argument #1 ($string) must be of
type string, int given...
```

The built-in function `trim()` is intended to only work with string data. When we pass an integer `$num` into the function, our program halts. When executing in this way, PHP will not intervene and change the type of `$num` – with predictably fatal results. Strict type checking only occurs on calls; operations are not affected:

```
declare(strict_types=1);

$str = "1";
var_dump($str * 2);

Output:
>int(2)
```